

N–1 Regional Failover Capacity, Worked Out by Hand

HOW MUCH CAPACITY EACH REGION MUST HOLD SO THAT LOSING ONE REGION LOSES NO USERS

What this document does. It takes one concrete topology — the *Nexus v7* service running active/active across 3 regions carrying 10,000 req/s total — and derives the per-region capacity floor that lets any *one* region fail with no capacity loss. The answer is not the naive T/N ; it is $T/(N-1)$ times a 1.15 headroom factor, giving $\sim 5,750$ req/s per region. We also show why a 2-region design forces each region to carry $2\times$ its baseline, and why more regions make over-provisioning cheaper. Every number is produced by the reference script (Appendix A, `m07_t03_failover_capacity_engine.py`).

The one idea. You must size each region for the world *after* a failure, not before it. When one of N regions is gone, the survivors split the *entire* load among $N-1$ of them — so the planning denominator is $N-1$, never N .

CONCEPTS COVERED, IN ORDER

baseline distribution T/N vs the survivor load $T/(N-1)$; the required-capacity formula with headroom; the Nexus 3-region case (5,750 req/s, 58% baseline, 87% on failover); why $N=2$ needs $2\times$ per region; and the efficiency trend as N grows.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The system under study	2
2	Step 1 — Baseline vs. the survivor load	2
3	Step 2 — The Nexus 3-region case	3
4	Step 3 — Why $N = 2$ costs $2\times$ per region	3
5	Step 4 — More regions, cheaper resilience	4
6	The argument, at a glance	4
6.1	Equation cheat-sheet	5
	Appendix A — Reference implementation	5

1 The system under study

A single region of Nexus can go dark for hours — a cloud control-plane incident, a bad deploy, a BGP misconfiguration. Serious production means multiple geographically independent regions and a routing layer that fails over in seconds. The capacity question is: how big must each region be so that losing one is invisible to users?

Quantity	Symbol	Value here
Total traffic across all regions	T	10,000 req/s
Number of regions	N	3 (active/active)
Headroom factor	h	1.15 (15% above the floor)
Baseline load per region	—	T/N
Survivor load (one region down)	—	$T/(N-1)$

In active/active, every region serves live traffic all the time (Nexus splits roughly 45/30/25 across US-East / EU-West / APAC). This is not a cold standby that warms up on failure — it is N live regions, any one of which can vanish.

Intuition

Three friends share a dinner bill equally: each pays 1/3. But they have agreed that if *any one* of them forgets their wallet, the other two must still cover the whole bill between them — so each must walk in carrying enough for *half*, not a third. Sizing for “everyone present” (T/N) is what you *use*; sizing for “one absent” ($T/(N-1)$) is what you must *provision*. The gap between those two is the cost of resilience.

2 Step 1 — Baseline vs. the survivor load

At baseline all N regions are up and traffic spreads evenly:

$$\text{baseline per region} = \frac{T}{N}.$$

But the instant one region fails, its share does not disappear — it re-routes to the survivors. Now T is carried by only $N-1$ regions:

$$\text{survivor load} = \frac{T}{N-1}.$$

The survivor load is the one that must fit inside each region’s capacity. Sizing to the baseline T/N guarantees an overflow the moment anything fails.

Required per-region capacity for N-1 survivability

$$\text{Required capacity per region} \geq \frac{T}{N-1} \times h, \quad h = 1.15.$$

The headroom factor $h = 1.15$ absorbs the queueing nonlinearity (Topic 1): you never want a survivor pinned at exactly 100%, because wait time explodes near saturation. The 15% buffer keeps even the post-failover state off the steep part of the latency curve.

3 Step 2 — The Nexus 3-region case

Plug in $N = 3$, $T = 10,000$

$$\begin{aligned}\text{baseline per region} &= \frac{T}{N} = \frac{10,000}{3} = 3,333 \text{ req/s}, \\ \text{survivor load} &= \frac{T}{N-1} = \frac{10,000}{2} = 5,000 \text{ req/s}, \\ \text{required capacity} &= \frac{T}{N-1} \times h = 5,000 \times 1.15 = \mathbf{5,750} \text{ req/s.} \quad \checkmark\end{aligned}$$

Now read off the two utilisations against that provisioned 5,750 req/s:

State	Load per region	Utilisation
Baseline (all 3 up)	3,333 req/s	$3333/5750 = \mathbf{58.0\%}$
Failover (one down)	5,000 req/s	$5000/5750 = \mathbf{87.0\%}$

Baseline utilisation is 58%, not 33%

The naive “3 regions, so each runs at $1/3 = 33\%$ ” is wrong for a *resilient* system. Because each region is provisioned for the $N-1$ world, its *baseline* utilisation is 58%, and when a sibling fails it climbs to 87% — hot, but inside capacity with headroom to spare. If you had sized for 33% baseline (capacity = 3,333), the first failover would demand 5,000 req/s from a region that can only do 3,333: instant overload, cascade, outage.

Watch out

The 87% failover utilisation is deliberately close to — but below — the load-shedding threshold from Topic 1 ($\rho^* \approx 0.835$ for the TTFT SLO). A region running at 87% during a failover is in mild SLO stress, which is acceptable for the minutes an outage lasts. Provision *less* headroom and a failover pushes survivors past the queueing knee; provision *more* and you pay for idle iron. $h = 1.15$ is the negotiated middle.

4 Step 3 — Why $N = 2$ costs $2\times$ per region

Repeat the arithmetic with two regions and the $N-1$ denominator collapses to 1:

$N = 2$ forces the full load onto one survivor

$$\begin{aligned}\text{baseline per region} &= \frac{T}{2} = 5,000 \text{ req/s}, \\ \text{survivor load} &= \frac{T}{N-1} = \frac{10,000}{1} = 10,000 \text{ req/s}, \\ \text{required capacity} &= 10,000 \times 1.15 = \mathbf{11,500} \text{ req/s}.\end{aligned}$$

The survivor must hold the *entire* T — exactly $2\times$ its 5,000 req/s baseline. Each of the two regions must therefore be provisioned at 11,500 req/s, versus 5,750 req/s for the three-region design:

$$\frac{\text{required cap}(N=2)}{\text{required cap}(N=3)} = \frac{11,500}{5,750} = 2.0 \times . \quad \checkmark$$

This is the precise sense in which two-region active/active is the “sensible minimum but expensive”: each region is sized for 100% of global traffic, so half your provisioned capacity sits idle waiting for the other region to fail.

5 Step 4 — More regions, cheaper resilience

The $N-1$ penalty is a fixed “one region’s worth” of extra load spread over the survivors. The more survivors there are, the thinner each one’s slice of that penalty:

N	Req. cap / region	Total provisioned	Overhead vs. T
2	11,500	23,000	130.0%
3	5,750	17,250	72.5%
4	3,833	15,333	53.3%
5	2,875	14,375	43.8%
8	1,643	13,143	31.4%

The over-provisioning trend

Total provisioned capacity is $N \cdot \frac{T}{N-1} \cdot h = \frac{N}{N-1} h T$. The factor $\frac{N}{N-1}$ shrinks from 2.0 ($N=2$) toward 1.0 as N grows, so the resilience overhead falls from 130% to just above the 15% headroom floor. More regions means the cost of surviving “any one down” is amortised over more iron — which is exactly why Nexus chose three regions rather than two.

Mini-example — the $(N-1)/N$ headroom view

The course also states the cost as “ $\sim (N-1)/N$ headroom.” Same fact, inverted: at $N=3$ the *useful* fraction of each region at failover is $\frac{N-1}{N} = \frac{2}{3} = 67\%$ of the provisioned $T/(N-1)$ baseline-equivalent — the other third is the failover reserve. As $N \rightarrow \infty$ that reserve fraction $\rightarrow 0$ and you approach paying only for the traffic you actually serve plus headroom.

6 The argument, at a glance

#	Step	Result
1	Plan for $N-1$	survivor load = $T/(N-1)$, not T/N
2	Nexus $N=3$	cap = $10000/2 \times 1.15 = 5,750$ req/s; 58%→87%
3	$N=2$ cost	survivor holds full T ; cap = 11,500 req/s ($2\times$)
4	Trend	overhead $\frac{N}{N-1}h$ falls: 130%→31% as $N:2\rightarrow 8$

6.1 Equation cheat-sheet

Baseline per region	T/N
Survivor load	$T/(N-1)$
Required capacity	$\frac{T}{N-1} \times h, \quad h = 1.15$
Baseline utilisation	$\frac{T/N}{\text{required cap}}$
Failover utilisation	$\frac{T/(N-1)}{\text{required cap}} = 1/h$
Total provisioned	$\frac{N}{N-1} h T$

Appendix A — Reference implementation

Every number above is produced by `m07_t03_failover_capacity_engine.py` (provided alongside this PDF). Run it to reproduce all figures; change T , N , or the headroom factor h to size your own topology.

```

1  """
2  Reference implementation for: Module 07 - Topic 3
3  "N-1 Regional Failover Capacity."
4
5  Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6  so the arithmetic in the document is exact and self-consistent.
7
8  Scenario: Nexus v7 runs active/active across N regions carrying total traffic
9  T = 10,000 req/s. To survive a single regional outage WITHOUT losing capacity,
10 each region must be sized for N-1 operation (one region down), times a 1.15
11 headroom factor. We compute per-region capacity for N = 2, 3, 5 and show the
12 utilisation a survivor hits when one region fails.
13
14 Convention: rates in requests/second.
15 """
16
17 # -----
18 # 0. The system under study.
19 # -----
20 T = 10000.0 # total traffic across all regions, req/s
21 headroom_factor = 1.15 # 15% headroom above the N-1 floor
22 N_main = 3 # Nexus choice: 3-region active/active
23
24 print("== 0. System under study ==")
25 print(f"total traffic T = {T:.0f} req/s")
26 print(f"headroom factor = {headroom_factor:.2f} (15% above N-1 floor)")
27 print(f"Nexus topology = {N_main}-region active/active")
28
29 # -----
30 # 1. The two distributions: baseline (all N up) vs N-1 (one down).
31 # -----
32 def baseline_per_region(T, N): return T / N
33 def n_minus_1_per_region(T, N): return T / (N - 1)
34 def required_capacity(T, N, h): return T / (N - 1) * h
35
36 print("\n== 1. Baseline vs N-1 load per region ==")
37 print(f"{'N':>3} {'baseline T/N':>14} {'N-1 load T/(N-1)':>18} "
38       f"{'required cap (x1.15)':>22}")
39 for N in [2, 3, 5]:
40     b = baseline_per_region(T, N)

```

```

41     nm1 = n_minus_1_per_region(T, N)
42     req = required_capacity(T, N, headroom_factor)
43     print(f"{N:>3} {b:>14.0f} {nm1:>18.0f} {req:>22.0f}")
44
45 # -----
46 # 2. The Nexus 3-region case, in detail.
47 # -----
48 print("\n== 2. Nexus: N = 3, T = 10,000 req/s ==")
49 N = N_main
50 b = baseline_per_region(T, N)
51 nm1 = n_minus_1_per_region(T, N)
52 req = required_capacity(T, N, headroom_factor)
53 print(f"baseline per region = T/N = 10000/3 = {b:.1f} req/s")
54 print(f"N-1 load per survivor = T/(N-1) = 10000/2 = {nm1:.0f} req/s")
55 print(f"required capacity = T/(N-1)*h = 5000*1.15 = {req:.0f} req/s")
56
57 # baseline utilisation against the required (provisioned) capacity:
58 u_baseline = b / req
59 print(f"baseline utilisation = {b:.1f}/{req:.0f} = {u_baseline*100:.1f}% "
60       f"(NOT 33.3% naive!)")
61
62 # when one region fails, the surviving regions each absorb T/(N-1):
63 u_failover = nm1 / req
64 print(f"failover utilisation = {nm1:.0f}/{req:.0f} = {u_failover*100:.1f}% "
65       f"(survivors run hot but within capacity)")
66
67 # -----
68 # 3. Why N=2 is the expensive case: each region needs 2x.
69 # -----
70 print("\n== 3. N=2 needs 2x per region ==")
71 req2 = required_capacity(T, 2, 1.0) # no headroom, pure N-1 for clarity
72 req2_h = required_capacity(T, 2, headroom_factor)
73 b2 = baseline_per_region(T, 2)
74 print(f"baseline per region (N=2) = T/2 = {b2:.0f} req/s")
75 print(f"N-1 per survivor (N=2) = T/(2-1) = {n_minus_1_per_region(T,2):.0f} req/s")
76 print(f"> survivor must hold the ENTIRE T = {n_minus_1_per_region(T,2):.0f} req/s "
77       f"= 2x its baseline {b2:.0f}")
78 print(f"required cap (x1.15) = {req2_h:.0f} req/s per region")
79
80 # ratio of required capacity N=2 vs N=3
81 print(f"\nrequired-capacity ratio N=2 / N=3 = "
82       f"{required_capacity(T,2,headroom_factor)/req:.2f}x "
83       f"(two regions cost ~2x the per-region iron of three)")
84
85 # -----
86 # 4. The efficiency trend: more regions => cheaper over-provisioning.
87 # -----
88 print("\n== 4. More regions = cheaper over-provisioning ==")
89 print(f"{N':>3} {'req cap/region':>15} {'total provisioned':>18} "
90       f"{'overhead vs T':>14}")
91 for N in [2, 3, 4, 5, 8]:
92     req = required_capacity(T, N, headroom_factor)
93     total = req * N
94     overhead = total / T - 1.0
95     print(f"{N:>3} {req:>15.0f} {total:>18.0f} {overhead*100:>13.1f}%")
96 print("As N grows, the N-1 'one-down' penalty is shared over more regions,")
97 print("so per-region over-provisioning shrinks toward just the headroom factor.")

```

— END OF DOCUMENT —